Venus DeviationBoundedOracle Audit Report

Please Note

1. The analysis of the Severity is purely based on the smart contracts mentioned in the Audit Scope and does not include any other potential contracts deployed by the Owner. No applications or operations were reviewed for Severity. No product code has been reviewed.
2. Due to the time limit, the audit team did not do much in-depth research on the business logic of the project. It is more about discovering issues in the smart contracts themselves.

Audit Period: 2026/04/14 - 2026/04/17 (YYYY/MM/DD)

Overall Risk: Medium

Project Name	Venus DeviationBoundedOracle Audit
Github	Repo1: https://github.com/VenusProtocol/oracle/pull/306 Commit: 7ad77af9973b7e52e587020eb8ab38ed11f5d95f Repo2: https://github.com/VenusProtocol/venus-protocol/pull/668 Commit: 95b8c6a0669227879bfc767f43c0ba9d49d48108 Fix PR: https://github.com/VenusProtocol/oracle/pull/308
Background	We have an audit request with the attached scope doc for Dual Price Oracle: https://www.notion.so/Audit-scope-DeviationBoundedOracle-Dual-Price-2026-04-13-Summary-341b4e5771bb8045a366de1c49d8acea?source=copy_link Fix: https://www.notion.so/183-348b4e5771bb802196eddd285d48f91f

Smart contracts list:

No.	File Name	Link	Verdict	Details
1	DeviationBoundedOracle.sol	https://github.com/VenusProtocol/oracle/blob/feat/vpd-893-bounded-price-oracle/contracts/DeviationBoundedOracle.sol	Medium	[M01]-Fixed [M02]-Ack [L01]-Fixed [I01]-Fixed [I02]-Fixed
2	IDeviationBoundedOracle.sol	https://github.com/VenusProtocol/oracle/blob/feat/vpd-893-bounded-price-oracle/contracts/interfaces/IDeviationBoundedOracle.sol	Green	
3	ComptrollerStorage.sol	https://github.com/VenusProtocol/venus-protocol/blob/feat/vpd-893-bounded-price-oracle/contracts/Comptroller/ComptrollerStorage.sol	Green	
4	Diamond.sol	https://github.com/VenusProtocol/venus-protocol/blob/feat/vpd-893-bounded-price-oracle/contracts/Comptroller/Diamond/Diamond.sol	Green	
5	ComptrollerInterface.sol	https://github.com/VenusProtocol/venus-protocol/blob/feat/vpd-893-bounded-price-oracle/contracts/Comptroller/ComptrollerLensInterface.sol	Green	
6	FacetBase.sol	https://github.com/VenusProtocol/venus-protocol/blob/feat/vpd-893-bounded-price-oracle/contracts/Comptroller/Diamond/facets/FacetBase.sol	Low	[L02]-Ack [I03]-Fixed
7	PolicyFacet.sol	https://github.com/VenusProtocol/venus-protocol/blob/feat/vpd-893-bounded-price-oracle/contracts/Comp	Low	[L02]-Ack

		troller/Diamond/facets/PolicyFacet.sol		
8	SetterFacet.sol	https://github.com/VenusProtocol/venus-protocol/blob/feat/vpd-893-bounded-price-oracle/contracts/Comptroller/Diamond/facets/SetterFacet.sol	**Green**	
9	ISetterFacet.sol	https://github.com/VenusProtocol/venus-protocol/blob/feat/vpd-893-bounded-price-oracle/contracts/Comptroller/Diamond/interfaces/ISetterFacet.sol	**Green**	
10	RewardFacet.sol	https://github.com/VenusProtocol/venus-protocol/blob/feat/vpd-893-bounded-price-oracle/contracts/Comptroller/Diamond/facets/RewardFacet.sol	**Informational**	**[I03]-Fixed**
11	VAIController.sol	https://github.com/VenusProtocol/venus-protocol/blob/feat/vpd-893-bounded-price-oracle/contracts/Tokens/VAI/VAIController.sol	**Low**	**[L01]-Fixed**
12	ComptrollerLens.sol	https://github.com/VenusProtocol/venus-protocol/blob/feat/vpd-893-bounded-price-oracle/contracts/Lens/ComptrollerLens.sol	**Green**	
13	SnapshotLens.sol	https://github.com/VenusProtocol/venus-protocol/blob/feat/vpd-893-bounded-price-oracle/contracts/Lens/SnapshotLens.sol	**Green**	
14	VenusLens.sol	https://github.com/VenusProtocol/venus-protocol/blob/feat/vpd-893-bo	**Low**	**[L02]-Ack**

		unded-price-oracle/contracts/Lens/VenusLens.sol		

Findings

[M01] Normal Price Recovery After a Crash Is Misidentified as a Pump, Preventing Protection Mode Exit	
Contract	DeviationBoundedOracle.sol
Severity Level	**Medium**

Description	**Description**
	_updateAndGetBoundedPrices executes _expandPriceWindow before _checkAndTriggerProtection. When a price crashes, _expandPriceWindow updates minPrice to the crash low first. Then _checkAndTriggerProtection uses this already-lowered minPrice to compute the pump threshold upperBound = minPrice * (1 + threshold). Any subsequent recovery above crashLow * (1 + threshold) is therefore misidentified as a pump, triggering protection and resetting lastProtectionTriggeredAt.
	
	JavaScript
	`// DeviationBoundedOracle – _updateAndGetBoundedPrices`
	`(uint128 updatedMin, uint128 updatedMax) = _expandPriceWindow(state, spot, asset); // window updated first`
	`bool protectionActive = _checkAndTriggerProtection(state, spot, asset); // detection uses updated minPrice`
	
	`// DeviationBoundedOracle – _exceedsDeviationThreshold`
	`uint256 upperBound = (uint256(minPrice) * (EXP_SCALE + threshold)) / EXP_SCALE; // pump baseline is minPrice`
	`uint256 lowerBound = (uint256(maxPrice) * (EXP_SCALE - threshold)) / EXP_SCALE; // crash baseline is maxPrice`
	`return (spot > upperBound || spot < lowerBound);`
	
	`// DeviationBoundedOracle – _checkAndTriggerProtection`
	`if (_exceedsDeviationThreshold(spot, state.minPrice, state.maxPrice, state.triggerThreshold)) {`
	`state.lastProtectionTriggeredAt = uint64(block.timestamp); // reset on every trigger`
	`...`
	`emit ProtectionTriggered(asset, spot, state.minPrice, state.maxPrice);`
	`return true;`
	`}`
	
	Concrete example: starting price 100, crashes to 65 → minPrice updated to 65, protection activates. Price recovers to 85: upperBound = 65 * 1.3 = 84.5, and 85 > 84.5 triggers pump detection, resetting lastProtectionTriggeredAt. As long as normal borrow/redeem activity keeps calling _updateProtectionStates, the cooldown timer is continuously reset and exitProtectionMode can never satisfy the time condition.

Impact:
Even after prices recover to reasonable levels following a crash, the protection mode cannot exit through the normal path because the cooldown timer is continuously reset by ordinary protocol activity. The only remedy is for the keeper to proactively call updateMinPrice to raise minPrice to a reasonable level. If keepers do not have an explicit SOP for this scenario or respond slowly, affected markets can be frozen indefinitely. Impacts all DBO-configured markets.

Recommendation

The root cause is the misclassification itself, not the cooldown reset logic — continuously resetting lastProtectionTriggeredAt on every deviation is intentional and correct for sustained manipulation scenarios. The actual problem is that price recovery after a crash is misidentified as a pump and unnecessarily resets the cooldown timer.

The fix should only reset the timer when prices make a new low (a genuine new threat) while protection is already active, and leave the timer running during price recovery:

```
JavaScript
  // DeviationBoundedOracle – _checkAndTriggerProtection
  if (_exceedsDeviationThreshold(spot, state.minPrice, state.maxPrice,
state.triggerThreshold)) {
      if (!state.currentlyUsingProtectedPrice) {
          state.currentlyUsingProtectedPrice = true;
          state.lastProtectionTriggeredAt = uint64(block.timestamp);
      } else if (spot < state.minPrice) {
          // already in protection – only reset timer if price makes a new
low (genuine new threat)
          state.lastProtectionTriggeredAt = uint64(block.timestamp);
      }
      emit ProtectionTriggered(asset, spot, state.minPrice, state.maxPrice);
      return true;
  }
```

Status

Fixed in [69b23a6]. The fix introduces a windowExpanded flag from _expandPriceWindow to distinguish genuine price extremes from recovery movements. The cooldown timer now resets only on first activation or when the window actually expands to a new low or high.

_expandPriceWindow now returns bool windowExpanded:

```
JavaScript
 function _expandPriceWindow(...) internal returns (uint128, uint128,
bool) {
      bool windowExpanded;
```

```
        if (spotU128 < currentMin) {
            _setMinPrice(state, asset, spotU128);
            currentMin = spotU128;
            windowExpanded = true;
        }
        if (spotU128 > currentMax) {
            _setMaxPrice(state, asset, spotU128);
            currentMax = spotU128;
            windowExpanded = true;
        }
        return (currentMin, currentMax, windowExpanded);
    }
```

_checkAndTriggerProtection resets timer only on first trigger or genuine window expansion:

JavaScript

```
 function _checkAndTriggerProtection(..., bool windowExpanded) internal
returns (bool) {
        if (_exceedsDeviationThreshold(...)) {
            bool enteringProtection = !state.currentlyUsingProtectedPrice;
            if (enteringProtection || windowExpanded) {
                state.lastProtectionTriggeredAt = uint64(block.timestamp);
            }
            if (enteringProtection) {
                state.currentlyUsingProtectedPrice = true;
            }
            emit ProtectionTriggered(...);
            return true;
        }
        if (state.currentlyUsingProtectedPrice) return true;
    }
```

[M02] Keeper can manipulate price window bounds to falsely trigger protection mode	
Contract	DeviationBoundedOracle.sol
Severity Level	**Medium**
Description	**Description** The DeviationBoundedOracle._validateAndUpdateBound() function allows a keeper to reduce or expand the price window with minimal validation via updateMinPrice/updateMaxPrice. The only constraints enforced are the following • MIN: newPrice < maxPrice AND newPrice <= currentSpot

- MAX: newPrice > minPrice AND newPrice >= currentSpot

Since there is no additional validation that the current spot price is itself legitimate, and there is essentially no limit on how far the new bound can be from spot, the lack of sufficient relative boundaries towards the spot price allows a keeper to push the price window to extreme values.

For example, the keeper can set minPrice = 1 or set maxPrice to type(uint128).max. per the above conditions

Vulnerable Code

```javascript
   /**
     * @notice Validates and applies a keeper-provided min or max price update
     * @param asset The underlying asset address
     * @param newPrice The new price value to set
     * @param boundType Whether this is a MIN or MAX bound update
     * @custom:error ZeroPriceNotAllowed if newPrice is zero
     * @custom:error MarketNotInitialized if the asset has not been initialized
     * @custom:error InvalidMinPrice if boundType is MIN and newPrice exceeds the current spot or is at or above maxPrice
     * @custom:error InvalidMaxPrice if boundType is MAX and newPrice is below the current spot or is at or below minPrice
     */
    function _validateAndUpdateBound(address asset, uint128 newPrice, PriceBoundType boundType) internal {
        ensureNonzeroAddress(asset);
        if (newPrice == 0) revert ZeroPriceNotAllowed();
        MarketProtectionState storage state = _ensureInitialized(asset);

        uint256 currentSpot = _fetchSpotPrice(asset);
        if (boundType == PriceBoundType.MIN) {
            if (newPrice >= state.maxPrice || uint256(newPrice) > currentSpot)
                revert InvalidMinPrice(asset, newPrice, currentSpot);
            _setMinPrice(state, asset, newPrice);
        } else if (boundType == PriceBoundType.MAX) {
            if (newPrice <= state.minPrice || uint256(newPrice) < currentSpot)
                revert InvalidMaxPrice(asset, newPrice, currentSpot);
            _setMaxPrice(state, asset, newPrice);
        }
    }
```

Impact

	An extremely wide window forces _exceedsDeviationThreshold to return true, given the threshold checks are easily satisfied, causing the protection mode to be triggered. Once triggered, _resolveBoundedPrices computes with the following mentioned rules ● collateralPrice = min(spot, windowMin) = the extremely low minPrice ● debtPrice = max(spot, windowMax) = the extremely high maxPrice This causes all collateral in the affected market to be valued near zero and all debt to be massively overvalued. Every account holding that asset appears to be in a shortfall, blocking borrows and redemptions.
Recommendation	● Consider introducing a maximum permissible distance between each bound and the current spot price. For example, enforce that newMin >= currentSpot * (1 - maxDrift) and newMax <= currentSpot * (1 + maxDrift), where maxDrift is a governance-configurable parameter. This can help reduce potential impact of a malicious/compromised keeper address. ● If not, enforce that the keeper role is fully trusted
Status	Acknowledged. The development team considers the finding invalid as it relates to liquidation — liquidation uses the LT+spot path and is unaffected. This is correct and the impact description was inaccurate on that point. However, the finding is maintained as a CF-path DoS concern. Setting minPrice = 1 or maxPrice = type(uint128).max blocks all borrowAllowed, redeemAllowed, and transferAllowed operations for all users in the affected market. Severity remains Low given ACM access control. No code fix was applied.

[L01] mintVAI Does Not Trigger DBO State Update, Price Manipulation Window History Lost	
Contract	VAIController.sol, DeviationBoundedOracle.sol
Severity Level	Low
Description	**Description** getMintableVAI is a public view function that calls getBoundedPricesView to obtain DBO prices. This call takes the pure view path (_computeBoundedPrices), which simulates protection based on the current spot price but does not write to storage, does not update minPrice/maxPrice, does not set currentlyUsingProtectedPrice, and does not update lastProtectionTriggeredAt. By contrast, borrowAllowed invokes _updateProtectionStates before its liquidity check, which calls updateProtectionState on each entered asset and durably writes the price window and protection state to DBO storage.

```javascript
  // VAIController – getMintableVAI
  function getMintableVAI(address minter) public view returns (uint256,
uint256) {
      ...
      IDeviationBoundedOracle boundedOracle =
comptroller.deviationBoundedOracle();
      // view path only – updateProtectionState is never called, storage is
never written
      (vars.collateralPriceMantissa, vars.debtPriceMantissa) =
boundedOracle.getBoundedPricesView(
          address(enteredMarkets[i])
      );
      ...
  }
  // DeviationBoundedOracle – _computeBoundedPrices (view path, no storage
writes)
  function _computeBoundedPrices(address vToken) internal view returns
(uint256 minPrice, uint256 maxPrice) {
      (minPrice, maxPrice) = _getCachedPrices(asset);
      if (minPrice != 0 && maxPrice != 0) return (minPrice, maxPrice);

      uint256 spot = _fetchSpotPrice(asset);
      MarketProtectionState storage state = assetProtectionConfig[asset];
      if (!state.isBoundedPricingEnabled) return (spot, spot);

      uint128 windowMin128 = spot < uint256(state.minPrice) ? spotU128 :
state.minPrice;
      uint128 windowMax128 = spot > uint256(state.maxPrice) ? spotU128 :
state.maxPrice;
      // simulates protection state but does not persist to storage
      bool shouldProtect = state.currentlyUsingProtectedPrice ||
          _exceedsDeviationThreshold(spot, windowMin128, windowMax128,
state.triggerThreshold);
      (minPrice, maxPrice) = _resolveBoundedPrices(shouldProtect, spot,
uint256(windowMin128), uint256(windowMax128));
  }
```

Because the mintVAI path never calls updateProtectionState, maxPrice in DBO storage is never updated through VAI minting activity. In a low-activity scenario where mintVAI is the dominant operation, the price window history of a pump-and-dump attack is never recorded:

1. Price pumps from 100 to 150; only mintVAI is active; DBO storage maxPrice remains 100
2. Price drops back to 110 (within 30% of the original window)

	3. View path: windowMax = max(110, 100) = 110; 110 does not exceed 100 × 1.3 = 130 → no protection → returns (110, 110) spot prices 4. Had updateProtectionState been called during the pump: maxPrice = 150; 110 < 150 × 0.7 = 105 → protection triggered → debtPrice = 150, VAI minting capacity severely restricted **Impact:** An attacker can conduct price manipulation exclusively via the mintVAI path, preventing DBO from recording the historical price extremes of the pump. After the price recovers, the protocol has no record of the manipulation, and users can mint VAI at spot prices — whereas the correct behavior would apply a conservative debt price anchored to the pump high. This creates a fundamental inconsistency with the borrowAllowed protection path and can be used to bypass DBO's pump-and-dump defense, resulting in excess VAI issuance.

Recommendation	Before calling getMintableVAI inside mintVAI, call updateProtectionState for all of the minter's entered assets to align the DBO state with the borrowAllowed path:
	
	JavaScript
	`function mintVAI(uint256 mintVAIAmount) external nonReentrant {`
	`  ...`
	`  IDeviationBoundedOracle dbo = comptroller.deviationBoundedOracle();`
	`  VToken[] memory assets = comptroller.getAssetsIn(minter);`
	`  for (uint256 i; i < assets.length; ++i) {`
	`  dbo.updateProtectionState(address(assets[i]));`
	`  }`
	`  (err, accountMintableVAI) = getMintableVAI(minter);`
	`  ...`
	`}`

Status	Fixed in 7357754. mintVAI now calls _updateProtectionStateForEnteredMarkets before computing the mintable VAI amount, ensuring DBO storage is updated through the VAI minting path and remains consistent with the borrow path. The pump-and-dump window history gap is resolved.

[L02] DBO Protection Mode Locks Redemption Path, Stripping Leveraged Users of Active Deleveraging Ability	
Contract	PolicyFacet.sol, FacetBase.sol, ComptrollerLens.sol
Severity Level	Low
Description	**Description**

redeemAllowed and transferAllowed use the CF path (USE_COLLATERAL_FACTOR), applying DBO's conservative bounded prices for health checks. When DBO protection mode is active, a user's CF-path health factor may show shortfall, causing redemptions and vToken transfers to be rejected.

```javascript
 // FacetBase — redeemAllowedInternal
  function redeemAllowedInternal(address vToken, address redeemer,
uint256 redeemTokens) internal returns (uint256) {
      ensureListed(getCorePoolMarket(vToken));
      if (!getCorePoolMarket(vToken).accountMembership[redeemer]) {
          return uint256(Error.NO_ERROR);
      }
      (Error err, , uint256 shortfall) =
getHypotheticalAccountLiquidityInternal(
          redeemer,
          VToken(vToken),
          redeemTokens,
          0,
          WeightFunction.USE_COLLATERAL_FACTOR  // DBO conservative price
path
      );
      ...
  }
```

In contrast, repayBorrowAllowed performs no price or liquidity check and is always executable:

```javascript
  // PolicyFacet — repayBorrowAllowed
  function repayBorrowAllowed(address vToken, address payer, address
borrower, uint256 repayAmount)
      external returns (uint256) {
      checkProtocolPauseState();
      checkActionPauseState(vToken, Action.REPAY);
      ensureListed(getCorePoolMarket(vToken));
      // No liquidity or price check
      return uint256(Error.NO_ERROR);
  }
```

Liquidation eligibility (liquidateBorrowAllowed) uses the LT path with spot oracle prices, completely bypassing DBO:

```javascript
  // ComptrollerLens - _calculateAccountPosition
  if (weightingStrategy == WeightFunction.USE_COLLATERAL_FACTOR) {
      vars.boundedOracle =
ComptrollerInterface(comptroller).deviationBoundedOracle();
  } else {
      // LT path - always spot; liquidation eligibility is determined here
      vars.spotOracle = ComptrollerInterface(comptroller).oracle();
  }
```

Attack path:
The following scenario illustrates how DBO protection mode can be weaponized:

```javascript
  Initial state:
    User: 1000 A collateral (spot=100), 700 B debt (B=100)
    CF=0.75, LT=0.85
    Liquidation threshold: spot(B) > 1000×100×0.85/700 ≈ 121.4

  Step 1: Attacker briefly pumps B to 140 (low cost)
    → DBO triggers: windowMax(B)=140, currentlyUsingProtectedPrice=true
    → B price immediately returns to 100

  Step 2: User's CF-path position:
    debtPrice(B) = max(100, windowMax=140) = 140 (inflated)
    sumCollateral = 1000 × 100 × 0.75 = 75,000
    sumDebt       = 700 × 140          = 98,000
    → CF shortfall; redeemAllowed rejected; user is frozen

  Step 3: User's normal defensive action (now blocked):
    Redeem partial A → sell for B → repay debt → reduce leverage
    → This path is blocked by DBO protection mode

  Step 4: Attacker pushes B's real price to 122
    LT path: sumCollateral = 85,000 < sumDebt = 700×122 = 85,400
    → LT shortfall; user is liquidatable
    → User could have self-rescued at B=110 but was denied this window
```

Impact:
When DBO protection mode is active, redeemAllowed blocks users from redeeming collateral even when their position is healthy at real spot prices. This removes a key defensive option — the redeem-then-repay deleveraging path — precisely when users need it most. An attacker can trigger DBO at low cost through a brief oracle price manipulation, and once the freeze is in place, push real prices toward the

liquidation threshold while the target user is unable to reduce their exposure. The victim can still repay debt directly if they hold the borrowed asset externally, or add more collateral via mintAllowed, but neither option is available to users who are fully leveraged within the protocol with no external reserves.

Recommendation

Add an LT-path fallback inside redeemAllowedInternal, but gate it on whether DBO protection mode is actually active. If no entered asset is currently in protection mode, the CF shortfall is genuine and the original behaviour is preserved. Only when DBO is responsible for the artificial shortfall does the fallback kick in — and even then, the redemption is only permitted if the LT path confirms the position is solvent at real market prices.

JavaScript

```
  if (shortfall != 0) {
      // Only apply LT fallback if DBO protection mode is actively causing
the shortfall.
      // Genuine CF shortfall (no protection active) should still be blocked.
      if (_isDboProtectionActive(redeemer)) {
          (Error ltErr, , uint256 ltShortfall) =
getHypotheticalAccountLiquidityInternalView(
              redeemer, VToken(vToken), redeemTokens, 0,
              WeightFunction.USE_LIQUIDATION_THRESHOLD
          );
          if (ltErr != Error.NO_ERROR || ltShortfall != 0) {
              return uint256(Error.INSUFFICIENT_LIQUIDITY);
          }
          // DBO-induced shortfall only; LT-healthy — allow redemption
          return uint256(Error.NO_ERROR);
      }
      return uint256(Error.INSUFFICIENT_LIQUIDITY);
  }

  // Helper: returns true if any entered asset is currently in DBO protection
mode
  function _isDboProtectionActive(address account) internal view returns
(bool) {
      IDeviationBoundedOracle dbo = deviationBoundedOracle;
      if (address(dbo) == address(0)) return false;
      VToken[] memory assets = accountAssets[account];
      for (uint256 i; i < assets.length; ++i) {
          address underlying = _getUnderlyingAsset(address(assets[i]));
          if (dbo.currentlyUsingProtectedPrice(underlying)) return true;
      }
      return false;
  }
```

Status	Acknowledged. The development team accepts the residual risk, noting that ResilientOracle's multi-source aggregation constrains oracle manipulation, affected users can still add collateral via mintAllowed, and active monitoring of protection triggers will be in place. The finding is retained in the report for transparency.

[I01] Window Re-seed on Re-enable Is Silent, Missing Boundary Update Events

Contract	DeviationBoundedOracle.sol
Severity Level	**Informational**

Description	**Description**
	When setAssetBoundedPricingEnabled re-enables bounded pricing, it writes directly to state.minPrice and state.maxPrice, bypassing the _setMinPrice/_setMaxPrice helper functions. Every other code path in the contract that modifies window bounds goes through these helpers which emit events before writing — this is the only exception.
	JavaScript
	`// DeviationBoundedOracle – setAssetBoundedPricingEnabled`
	`if (enabled) {`
	`uint128 spotU128 = _safeToUint128(_fetchSpotPrice(asset));`
	`state.minPrice = spotU128; // direct write, no event`
	`state.maxPrice = spotU128; // direct write, no event`
	`}`
	Every other code path in the contract that modifies window bounds goes through the helper functions, which emit the event before writing storage:
	JavaScript
	`// DeviationBoundedOracle – _setMinPrice / _setMaxPrice`
	`function _setMinPrice(MarketProtectionState storage state, address asset, uint128 newMin) internal {`
	`emit MinPriceUpdated(asset, state.minPrice, newMin);`
	`state.minPrice = newMin;`
	`}`
	Impact:
	Off-chain systems (keepers, risk monitors) that reconstruct window state from events will hold stale minPrice / maxPrice values after a re-enable. This causes checkAndGetWindowDrift to produce inaccurate drift signals, potentially triggering unnecessary keeper updates or missing necessary boundary corrections. The incomplete event history also reduces on-chain auditability.

Recommendation	Replace direct storage writes with the existing helper functions:
	
	JavaScript
	`// DeviationBoundedOracle – setAssetBoundedPricingEnabled`
	`if (enabled) {`
	`  uint256 spot = _fetchSpotPrice(asset);`
	`  if (spot == 0) revert ZeroPriceNotAllowed();`
	`  uint128 spotU128 = _safeToUint128(spot);`
	`  _setMinPrice(state, asset, spotU128);`
	`  _setMaxPrice(state, asset, spotU128);`
	`}`

Status	Fixed in 058d651. setAssetBoundedPricingEnabled's re-enable branch now routes the window re-seed through _setMinPrice and _setMaxPrice, ensuring MinPriceUpdated and MaxPriceUpdated events are emitted alongside the storage write, consistent with all other window-update paths in the contract.

[I02] updateProtectionState Has No Access Control, Allowing Anyone to Reset the Protection Cooldown Timer

Contract	DeviationBoundedOracle.sol
Severity Level	**Informational**

Description	**Description**
	updateProtectionState is an external function with no access control — any address can call it at any time. Internally it calls _checkAndTriggerProtection, which resets lastProtectionTriggeredAt whenever the current price still exceeds the deviation threshold. Combined with exitProtectionMode's cooldown check, anyone can prevent keepers from exiting protection mode by calling this function repeatedly while prices remain slightly outside the threshold.
	
	JavaScript
	`// DeviationBoundedOracle – updateProtectionState`
	`function updateProtectionState(address vToken) external {`
	`  _updateAndGetBoundedPrices(vToken); // no access control`
	`}`
	
	`// DeviationBoundedOracle – exitProtectionMode`
	`if (block.timestamp < uint256(state.lastProtectionTriggeredAt) +`
	`uint256(state.cooldownPeriod)) {`

```
        revert CooldownNotElapsed(asset, state.lastProtectionTriggeredAt,
state.cooldownPeriod);
    }
```

Impact:
Any user can call updateProtectionState to reset the cooldown timer, which may cause the actual exit timing to deviate from the administrator's expectations. As a result, calls to exitProtectionMode may fail due to the cooldown being repeatedly extended.

Recommendation	If there is no specific requirement for public access, it is recommended to restrict the caller permissions, for example by allowing only the Comptroller contract to invoke it. If public access is required (e.g., for gas efficiency or cooperative design), this behavior should be clearly documented at a minimum.
Status	Fixed in f0de73d. The permissionless design is intentional. The team has updated the natspec of updateProtectionState to explicitly document that anyone can call it, for gas efficiency and to ensure consistent bounded price reads within a transaction. No code-level access restriction was applied.

[I03] RewardFacet Implicitly Inherits DBO Behavior With No Documentation

Contract	RewardFacet.sol, FacetBase.sol
Severity Level	**Informational**
Description	**Description** RewardFacet was not explicitly modified in this PR, but its inheritance chain is RewardFacet → XVSRewardsHelper → FacetBase. FacetBase.getHypotheticalAccountLiquidityInternal was changed from view to non-view in this PR, and the CF path now calls _updateProtectionStates before computing liquidity using DBO bounded prices. Since claimVenus calls this function with USE_COLLATERAL_FACTOR, it has implicitly picked up DBO behavior without any explicit change or documentation.

```javascript
// RewardFacet — claimVenus
(, , uint256 shortfall) = getHypotheticalAccountLiquidityInternal(
    holder,
    VToken(address(0)),
    0,
    0,
```

```
      WeightFunction.USE_COLLATERAL_FACTOR // now triggers
_updateProtectionStates via FacetBase
  );
  uint256 returnAmount = grantXVSInternal(holder, value, shortfall,
collateral);
  // shortfall > 0 → XVS not granted as collateral
```

When DBO protection mode is active, users may show CF-path shortfall even if their position is healthy at spot prices, causing shortfall > 0 and blocking XVS rewards from being granted as collateral.

Impact:
During DBO protection mode, users with genuinely healthy positions (at real market prices) cannot receive XVS rewards as collateral through claimVenus. Users relying on XVS rewards to maintain their collateral health ratio are silently affected. This is an undocumented behavioral change that may surprise users and integrators.

Recommendation

Document this behavioral change explicitly in the PR description and in RewardFacet's natspec. If XVS reward distribution should not be affected by DBO protection mode, consider switching the health check to the LT path:

JavaScript
```
  (, , uint256 shortfall) = getHypotheticalAccountLiquidityInternalView(
      holder, VToken(address(0)), 0, 0,
      WeightFunction.USE_LIQUIDATION_THRESHOLD
  );
```

Status

Fixed in a066e96. The development team added a natspec comment to claimVenus explicitly documenting that the shortfall probe uses the CF path with DBO bounded prices. Under protection mode, XVS may not be granted as collateral even for positions that are healthy at spot prices. No logic change was applied.